2600
2600, école de cybersécurité

MODULE
Noyau Linux

TP 04

Programmation du noyau Linux

BPF-LSM : miniEDR

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

Sommaire

Objectifs 3
Phase 1 — Environnement et premier programme BPF (1h) 4
1.1 : Recompiler le noyau avec CONFIG_BPF_LSM 4
1.2 : Installer la toolchain dans QEMU 5
1.3 : Structure du projet 5
1.4 : Fichiers fournis 5
`Makefile` 5
`miniedr.h` — structures partagées 6
`miniedr.bpf.c` — squelette BPF 7
`miniedr.c` — squelette daemon userspace 8
`jailer.c` — squelette outil Phase 3 10
1.5 : Compilation et premier test 11
Phase 2 — Logger d'activité (1h30) 12
2.1 : Passage en mode sleepable 12
2.2 : Implémenter les TODOs Phase 2 12
2.3 : Test 12
2.4 : Questions 13
Phase 3 — Emprisonnement de processus (2h) 13
3.1 : Épinglage de la map 13
3.2 : Implémenter le hook de prison 13
3.3 : Implémenter jailer 14
3.4 : Test d'emprisonnement 14
3.5 : Questions 14
Bonus — Bloquer le chargement de modules 15
Ressources 15

Objectifs

L'objectif de ce TP est de construire un **miniEDR** (Endpoint Detection and Response) utilisant BPF-LSM pour surveiller l'activité d'un système Linux et emprisonner des processus.

Le projet se divise en trois phases progressives :

- **Phase 1 (1h)** — Environnement et premier programme BPF-LSM
- **Phase 2 (1h30)** — Logger d'activité via ring buffer
- **Phase 3 (2h)** — Emprisonnement de processus

Vous apprendrez :

- Comment recompiler un noyau avec `CONFIG_BPF_LSM`.
- Comment écrire un programme BPF avec libbpf et CO-RE.
- Comment attacher un programme BPF à un hook LSM.
- Comment transmettre des événements du noyau vers l'espace utilisateur.
- Comment utiliser des BPF maps pour implémenter une politique de sécurité.



Phase 1 — Environnement et premier programme BPF (1h)

1.1 : Recompiler le noyau avec CONFIG_BPF_LSM

Votre noyau QEMU doit avoir les options suivantes activées. Dans `make menuconfig` :

```
code

General setup
  └─ BPF subsystem
       ├─ Enable bpf() system call                (CONFIG_BPF_SYSCALL=y)
       └─ Enable BPF LSM Instrumentation          (CONFIG_BPF_LSM=y)

Kernel hacking
  └─ Compile-time checks and compiler options
       └─ Debug information
            └─ Generate BTF type information      (CONFIG_DEBUG_INFO_BTF=y)
```

Ou directement dans `.config` :

```
code

CONFIG_BPF=y
CONFIG_BPF_SYSCALL=y
CONFIG_BPF_LSM=y
CONFIG_DEBUG_INFO_BTF=y
```

Puis activer BPF-LSM dans la liste des LSM (ligne de commande noyau ou `CONFIG_LSM`) :

```
code

lsm=yama,bpf
```

Recompiler et redémarrer QEMU. Vérifier que BPF-LSM est actif :

```
shell

$ cat /sys/kernel/security/lsm
yama,bpf
```

1.2 : Installer la toolchain dans QEMU

```shell
# Alpine Linux
$ apk add clang llvm bpftool libbpf-dev linux-headers

# Debian/Ubuntu (si autre distrib)
$ apt install clang llvm bpftool libbpf-dev linux-tools-$(uname -r)
```

Vérifier :

```shell
$ clang --version          # >= 12
$ bpftool version
$ ls /sys/kernel/btf/vmlinux   # doit exister (CONFIG_DEBUG_INFO_BTF)
```

1.3 : Structure du projet

Créer le répertoire de travail :

```code
miniedr/
├── Makefile          (fourni)
├── miniedr.h         (fourni — structures partagées BPF/userspace)
├── miniedr.bpf.c     (squelette fourni — programmes BPF)
├── miniedr.c         (squelette fourni — daemon userspace)
└── jailer.c          (squelette fourni — outil Phase 3)
```

1.4 : Fichiers fournis

Makefile

```makefile
CLANG     ?= clang
CC        ?= gcc
BPFTOOL   ?= bpftool
ARCH      := $(shell uname -m | sed 's/x86_64/x86/')

BPF_CFLAGS := -g -O2 -target bpf -D__TARGET_ARCH_$(ARCH) \
              -I/usr/include -I.
```

```
CFLAGS     := -g -O2 -Wall -I.

all: vmlinux.h miniedr jailer

vmlinux.h:
    $(BPFTOOL) btf dump file /sys/kernel/btf/vmlinux format c > $@

miniedr.bpf.o: miniedr.bpf.c vmlinux.h miniedr.h
    $(CLANG) $(BPF_CFLAGS) -c $< -o $@

miniedr.skel.h: miniedr.bpf.o
    $(BPFTOOL) gen skeleton $< > $@

miniedr: miniedr.c miniedr.skel.h miniedr.h
    $(CC) $(CFLAGS) -o $@ $< -lbpf -lelf -lz

jailer: jailer.c miniedr.h
    $(CC) $(CFLAGS) -o $@ $< -lbpf

clean:
    rm -f vmlinux.h *.o *.skel.h miniedr jailer

.PHONY: all clean
```

`miniedr.h` — structures partagées

```c
#pragma once
#include <linux/types.h>

#define MAX_PATH_LEN    256
#define TASK_COMM_LEN    16

/* Types d'événements */
#define EVENT_EXEC        1
#define EVENT_FILE_OPEN   2

/* Flags de politique d'emprisonnement */
#define JAIL_BLOCK_ETC   (1U << 0)   /* interdire /etc/  */
#define JAIL_BLOCK_PROC  (1U << 1)   /* interdire /proc/ */
#define JAIL_BLOCK_ROOT  (1U << 2)   /* interdire /root/ */

struct event {
    __u32 pid;
    __u32 ppid;
    __u32 uid;
    __u32 type;                  /* EVENT_EXEC ou EVENT_FILE_OPEN */
    char  comm[TASK_COMM_LEN];
    char  path[MAX_PATH_LEN];
};

struct jail_policy {
    __u32 flags;                 /* ORed JAIL_BLOCK_* */
};
```

`miniedr.bpf.c` — squelette BPF

```c
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>
#include "miniedr.h"

char LICENSE[] SEC("license") = "GPL";

/* ─── Maps ──────────────────────────────────────────── */

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 24);   /* 16 MiB */
} events SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 256);
    __type(key,   __u32);
    __type(value, struct jail_policy);
} jail_map SEC(".maps");

/* ─── Hook : ouverture de fichier ──────────────────
 *
 * NOTE Phase 2 : changer "lsm/file_open" en "lsm.s/file_open"
 *                pour activer le helper bpf_d_path().
 */
SEC("lsm/file_open")
int BPF_PROG(file_open_hook, struct file *file)
{
    u32 pid = bpf_get_current_pid_tgid() >> 32;

    /* ── Phase 1 ──────────────────────────────────────
     *
     * TODO :
     *   Récupérer le nom du processus courant avec bpf_get_current_comm()
     *   et afficher pid + comm avec bpf_printk().
     *
     *   Vérification :
     *     $ sudo cat /sys/kernel/debug/tracing/trace_pipe
     *
     *   Ressources :
     *     cours kernel_04 — "Structure d'un programme BPF"
     *     man bpf-helpers → bpf_get_current_comm, bpf_printk
     */

    /* ── Phase 2 ──────────────────────────────────────
     *
     * TODO (après Phase 1) :
     *   1. Changer la section en "lsm.s/file_open" (sleepable).
     *   2. Réserver un slot dans events : bpf_ringbuf_reserve(&events, ...).
     *   3. Remplir les champs :
     *        e->type = EVENT_FILE_OPEN
     *        e->pid  = pid
     *        e->uid  = (u32) bpf_get_current_uid_gid()
     *        e->comm via bpf_get_current_comm()
     *        e->ppid via bpf_get_current_task_btf() + BPF_CORE_READ()
```

```
     *   4. Récupérer le chemin :
     *        bpf_d_path(&file->f_path, e->path, MAX_PATH_LEN)
     *   5. Soumettre avec bpf_ringbuf_submit().
     *
     *   Ressources :
     *     cours kernel_04 — "Ring Buffer" + "Hooks EDR : accès aux fichiers"
     *     man bpf-helpers → bpf_d_path, bpf_ringbuf_reserve
     */

    /* ── Phase 3 ──────────────────────────────────────── */
     *
     * TODO (après Phase 2) :
     *   1. Chercher pid dans jail_map :
     *        struct jail_policy *p = bpf_map_lookup_elem(&jail_map, &pid);
     *        if (!p) return 0;
     *   2. Récupérer le chemin (cf. Phase 2).
     *   3. Pour chaque flag actif, vérifier si le chemin commence
     *      par le préfixe interdit et retourner -EPERM.
     *        JAIL_BLOCK_ETC   → "/etc/"  (5 premiers caractères)
     *        JAIL_BLOCK_PROC  → "/proc/" (6 premiers caractères)
     *        JAIL_BLOCK_ROOT  → "/root/" (6 premiers caractères)
     *      Comparaison : bpf_strncmp(path, n, prefix) (Linux >= 5.17)
     *      Ou boucle bornée sur les premiers caractères.
     *
     *   Ressources :
     *     cours kernel_04 — "Emprisonnement : le hook"
     *     man bpf-helpers → bpf_strncmp, bpf_map_lookup_elem
     */

    return 0;
}

/* ─── Hook : exécution de binaire ──────────────────  */

SEC("lsm/bprm_check_security")
int BPF_PROG(exec_hook, struct linux_binprm *bprm)
{
    /* ── Phase 2 ──────────────────────────────────────── */
     *
     * TODO :
     *   1. Réserver un slot dans events.
     *   2. Remplir : type = EVENT_EXEC, pid, ppid, uid, comm.
     *      Pour ppid :
     *        struct task_struct *t = (struct task_struct *)
     *            bpf_get_current_task_btf();
     *        e->ppid = BPF_CORE_READ(t, real_parent, tgid);
     *   3. Laisser e->path vide (execve : le binaire est dans bprm->filename,
     *      mais sa lecture est optionnelle pour cette phase).
     *   4. Soumettre.
     *
     *   Ressources :
     *     cours kernel_04 — "Ring Buffer" + "Informations sur le processus"
     */

    return 0;
}
```

`miniedr.c` — squelette daemon userspace

```
c
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <bpf/libbpf.h>
#include "miniedr.skel.h"
#include "miniedr.h"

static volatile int running = 1;
static void sig_handler(int sig) { running = 0; }

/* ── Phase 2 : callback appelé pour chaque événement du ring buffer ── */

static int handle_event(void *ctx, void *data, size_t sz)
{
    struct event *e = data;

    /*
     * TODO (Phase 2) :
     *   Afficher les champs de l'événement selon son type.
     *
     *   Format suggéré :
     *     [EXEC]      pid=1234 ppid=1 uid=0 comm=bash
     *     [FILE_OPEN] pid=1234 ppid=1 uid=1000 comm=cat path=/etc/passwd
     *
     *   Ressource : miniedr.h → définitions EVENT_EXEC, EVENT_FILE_OPEN
     */

    return 0;
}

int main(void)
{
    struct miniedr *skel;
    struct ring_buffer *rb = NULL;
    int err;

    /* Charger et attacher les programmes BPF */
    skel = miniedr__open_and_load();
    if (!skel) {
        fprintf(stderr, "Erreur : impossible de charger le BPF\n");
        return 1;
    }

    err = miniedr__attach(skel);
    if (err) {
        fprintf(stderr, "Erreur : impossible d'attacher les hooks\n");
        goto cleanup;
    }

    /* ── Phase 3 : épingler jail_map pour y accéder depuis jailer ──────
     *
     * TODO (Phase 3) :
     *   Épingler la map dans le pseudo-FS BPF :
     *     mkdir -p /sys/fs/bpf/miniedr
     *     bpf_map__pin(skel->maps.jail_map, "/sys/fs/bpf/miniedr/jail_map")
     *
     *   Ressource : libbpf API → bpf_map__pin
     */

    /* ── Phase 2 : créer le ring buffer ── */
```

```
    /*
     * TODO (Phase 2) :
     *   rb = ring_buffer__new(
     *       bpf_map__fd(skel->maps.events), handle_event, NULL, NULL);
     *   if (!rb) { perror("ring_buffer"); err = -1; goto cleanup; }
     *
     *   Ressource : libbpf API → ring_buffer__new, ring_buffer__poll
     */

    signal(SIGINT,  sig_handler);
    signal(SIGTERM, sig_handler);
    printf("miniEDR démarré (pid=%d). Ctrl-C pour arrêter.\n", getpid());

    while (running) {
        if (rb)
            ring_buffer__poll(rb, 100 /* ms */);
        else
            pause();
    }

cleanup:
    ring_buffer__free(rb);

    /* ── Phase 3 : dépingler la map ── */
    /* bpf_map__unpin(skel->maps.jail_map, "/sys/fs/bpf/miniedr/jail_map"); */

    miniedr__destroy(skel);
    return err < 0 ? -err : 0;
}
```

`jailer.c` — squelette outil Phase 3

c

```
/*
 * jailer — configure la politique d'emprisonnement d'un processus
 *
 * Usage : ./jailer <pid> [etc] [proc] [root]
 * Exemple : ./jailer 1234 etc proc
 *           → interdit au pid 1234 l'accès à /etc/ et /proc/
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <bpf/bpf.h>
#include "miniedr.h"

int main(int argc, char **argv)
{
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <pid> [etc] [proc] [root]\n", argv[0]);
        return 1;
    }

    __u32 target_pid = (__u32)atoi(argv[1]);
    __u32 flags      = 0;

    for (int i = 2; i < argc; i++) {
```

```
        if (strcmp(argv[i], "etc")  == 0) flags |= JAIL_BLOCK_ETC;
        if (strcmp(argv[i], "proc") == 0) flags |= JAIL_BLOCK_PROC;
        if (strcmp(argv[i], "root") == 0) flags |= JAIL_BLOCK_ROOT;
    }

    struct jail_policy policy = { .flags = flags };

    /*
     * TODO (Phase 3) :
     *   1. Ouvrir la map épinglée par miniedr :
     *        int map_fd = bpf_obj_get("/sys/fs/bpf/miniedr/jail_map");
     *        if (map_fd < 0) { perror("bpf_obj_get"); return 1; }
     *
     *   2. Insérer la politique dans la map :
     *        bpf_map_update_elem(map_fd, &target_pid, &policy, BPF_ANY);
     *
     *   3. Afficher un message de confirmation.
     *
     *   Ressource : man bpf_obj_get, man bpf_map_update_elem
     *               libbpf API → https://libbpf.readthedocs.io
     */

    return 0;
}
```

1.5 : Compilation et premier test

```shell
$ cd miniedr
$ make
```

Le build enchaîne : 1. `bpftool btf dump …` → génère `vmlinux.h` 2. `clang -target bpf …` → compile `miniedr.bpf.o` 3. `bpftool gen skeleton …` → génère `miniedr.skel.h` 4. `gcc …` → compile `miniedr` et `jailer`

Implémenter le TODO Phase 1 dans `file_open_hook`, puis :

```shell
$ sudo ./miniedr &
$ cat /etc/hostname
$ sudo cat /sys/kernel/debug/tracing/trace_pipe
# → doit afficher les lignes bpf_printk pour chaque open
```

Phase 2 — Logger d'activité (1h30)

2.1 : Passage en mode sleepable

Pour pouvoir utiliser `bpf_d_path()`, le hook `file_open_hook` doit être sleepable. Modifier la section dans `miniedr.bpf.c` :

```c
// Avant :
SEC("lsm/file_open")

// Après :
SEC("lsm.s/file_open")
```

`bpf_d_path` résout le chemin complet d'un fichier depuis un `struct path *`. Elle n'est disponible que dans les programmes sleepable.

2.2 : Implémenter les TODOs Phase 2

Implémenter dans `miniedr.bpf.c` : - le corps de `file_open_hook` (ring buffer + bpf_d_path) - le corps de `exec_hook` (ring buffer + ppid via CO-RE)

Implémenter dans `miniedr.c` : - la fonction `handle_event` (affichage des événements) - la création du ring buffer dans `main`

2.3 : Test

```shell
$ sudo ./miniedr
miniEDR démarré (pid=…). Ctrl-C pour arrêter.

# Dans un autre terminal :
$ ls /tmp
$ cat /etc/hostname
$ echo test > /tmp/test.txt

# Sortie attendue dans miniEDR :
[FILE_OPEN] pid=1234 ppid=1 uid=1000 comm=ls     path=/tmp
[FILE_OPEN] pid=1235 ppid=1 uid=1000 comm=cat    path=/etc/hostname
[EXEC]      pid=1236 ppid=1 uid=1000 comm=cat
```

2.4 : Questions

1. Pourquoi certains `FILE_OPEN` apparaissent avec un chemin vide ou tronqué ? Dans quels cas `bpf_d_path` peut-elle échouer ?
2. Observer les événements générés par `sudo ./miniedr` lui-même au démarrage. Qu'est-ce qui explique ce comportement ?
3. Comment filtrer les événements pour n'afficher que ceux du processus `bash` ? Ajouter ce filtre côté BPF (dans le programme BPF, pas en userspace) pour éviter le bruit.

Phase 3 — Emprisonnement de processus (2h)

3.1 : Épinglage de la map

Pour que `jailer` puisse écrire dans `jail_map` sans être le même processus que `miniedr`, la map doit être **épinglée** dans le pseudo-système de fichiers BPF.

Implémenter le TODO Phase 3 dans `miniedr.c` (section épinglage) :

```c
mkdir("/sys/fs/bpf/miniedr", 0700);
bpf_map__pin(skel->maps.jail_map, "/sys/fs/bpf/miniedr/jail_map");
```

> *Note : `bpf_map__pin` retourne 0 en cas de succès, négatif sinon.*

3.2 : Implémenter le hook de prison

Implémenter le TODO Phase 3 dans `file_open_hook` de `miniedr.bpf.c`.

Points d'attention : - La recherche dans `jail_map` doit se faire sur le PID courant. - `bpf_d_path` peut échouer (pipe, socket...) — vérifier le code de retour. - `bpf_strncmp(s, n, lit)` compare `s` (borné à `n` octets) à `lit` (chaîne littérale). Disponible depuis Linux 5.17. Pour les noyaux antérieurs, utiliser une boucle bornée :

```c
```

```
// Équivalent manuel de bpf_strncmp(path, 5, "/etc/")
static __always_inline int starts_with_etc(const char *p)
{
    return p[0]=='/' && p[1]=='e' && p[2]=='t' && p[3]=='c' && p[4]=='/';
}
```

3.3 : Implémenter jailer

Implémenter le TODO dans `jailer.c` : - `bpf_obj_get` pour ouvrir la map épinglée - `bpf_map_update_elem` pour insérer la politique

3.4 : Test d'emprisonnement

```shell
# Terminal 1 : démarrer miniEDR
$ sudo ./miniedr

# Terminal 2 : lancer un shell cible
$ bash
$ echo $$          # noter le PID, ex: 5678
$ cat /etc/passwd  # doit fonctionner

# Terminal 3 : emprisonner le shell
$ sudo ./jailer 5678 etc proc
PID 5678 emprisonné (flags=0x3)

# Retour dans Terminal 2 :
$ cat /etc/passwd
cat: /etc/passwd: Operation not permitted   ← -EPERM !
$ ls /proc/1
ls: /proc/1: Operation not permitted        ← -EPERM !
$ ls /tmp
[liste normale]                             ← /tmp non bloqué
```

3.5 : Questions

1. Lancer `sudo cat /etc/passwd` depuis le shell emprisonné. Pourquoi fonctionne-t-il ou non ? Que révèle ce comportement sur la granularité PID vs cgroup ?
2. Modifier `file_open_hook` pour hériter la politique du parent : si `jail_map[ppid]` existe et que `jail_map[pid]` n'existe pas, appliquer la politique du parent.
3. Que se passe-t-il si `miniedr` est tué alors qu'un shell est emprisonné ? Expliquer pourquoi.

Bonus — Bloquer le chargement de modules

Implémenter un nouveau hook dans `miniedr.bpf.c` qui, si le processus appelant `insmod` est emprisonné, retourne `-EPERM` :

```c
SEC("lsm/kernel_module_request")
int BPF_PROG(kmod_hook, char *kmod_name)
{
    /*
     * TODO :
     *   1. Vérifier si le PID courant est dans jail_map.
     *   2. Si oui, logguer le nom du module et retourner -EPERM.
     *   3. Sinon, retourner 0.
     */
    return 0;
}
```

Test :

```shell
# Depuis un shell emprisonné :
$ sudo insmod some_module.ko
insmod: ERROR: could not insert module: Operation not permitted

# Depuis un shell non emprisonné :
$ sudo insmod some_module.ko
[fonctionne normalement]
```

Ressources

- **Cours** : `kernel_04.md` — toutes les slides sont référencées dans les TODOs
- **BPF helpers** : `man bpf-helpers` (ou `bpftool feature`)
- **libbpf API** : https://libbpf.readthedocs.io/en/latest/api.html
- **Exemples BPF-LSM** : `samples/bpf/` et `tools/testing/selftests/bpf/progs/` dans les sources noyau Linux
- **Référence hooks LSM** : `include/linux/lsm_hook_defs.h` dans les sources noyau
- **BTF/CO-RE** : https://nakryiko.com/posts/bpf-core-reference-guide/